

0/1 Knapsack Problem

Dynamic Programming Approach — Algorithm & Implementation in C

1. Aim

To design and implement a C program that solves the 0/1 Knapsack problem using Dynamic Programming (DP), computing the maximum total value that can be packed into a knapsack of fixed capacity without exceeding its weight limit.

2. Problem Description

Given:

1. n items, each with a weight $w[i]$ and a value $v[i]$.
2. A knapsack with maximum weight capacity W .
3. Each item can be taken at most once (0 or 1) — it cannot be fractioned.

Goal: Select a subset of items such that:

Maximise: sum of $v[i]$ for all selected items

Subject to: sum of $w[i]$ for selected items $\leq W$

The **0/1** constraint means each item is either fully included (1) or excluded (0). This distinguishes the problem from the Fractional Knapsack where partial items are allowed.

3. Algorithm

3.1 DP Recurrence

Define $K[i][w]$ = maximum value achievable using the first i items with capacity w .

$$K[i][w] = \begin{cases} K[i-1][w] & \text{if } w[i] > w \\ \max(K[i-1][w], K[i-1][w-w[i]] + v[i]) & \text{otherwise} \end{cases}$$

Base case: $K[0][w] = 0$ for all w (no items $\rightarrow$ zero value); $K[i][0] = 0$ for all i (zero capacity $\rightarrow$ zero value).

3.2 Pseudocode

```
KNAPSACK_DP(weights[], values[], n, W)

// Step 1: Initialise DP table
for i = 0 to n do
    for w = 0 to W do
        K[i][w] = 0
    end for
end for

// Step 2: Fill table row by row
```

```

for i = 1 to n do
  for w = 0 to W do
    if weights[i] > w then
      K[i][w] = K[i-1][w]          // item i cannot fit
    else
      K[i][w] = max( K[i-1][w],    // exclude item i
                    K[i-1][w - weights[i]] + values[i] ) // include
    end if
  end for
end for

// Step 3: Traceback to find selected items
w = W
for i = n downto 1 do
  if K[i][w] != K[i-1][w] then
    print "Select item", i
    w = w - weights[i]
  end if
end for

return K[n][W]    // maximum value

```

3.3 Step-by-Step Execution

1. Initialise a 2-D table K of size $(n+1) \times (W+1)$ with all zeros.
2. For each item i from 1 to n (outer loop):
3. For each capacity w from 0 to W (inner loop):
4. If item i is too heavy ($w[i] > w$): copy the value from the row above $\rightarrow K[i][w] = K[i-1][w]$.
5. Otherwise: take the best of excluding item i ($K[i-1][w]$) or including it ($K[i-1][w-w[i]] + v[i]$).
6. After filling, $K[n][W]$ holds the maximum value.
7. Traceback from $K[n][W]$: if $K[i][w]$ differs from $K[i-1][w]$, item i was selected; subtract its weight and continue.

4. Worked Example

4.1 Input Data

Item	Weight ($w[i]$)	Value ($v[i]$)	Description
1	2	3	Item 1
2	3	4	Item 2
3	4	5	Item 3
4	5	6	Item 4

Knapsack Capacity $W = 8$

4.2 DP Table $K[i][w]$

Each cell $K[i][w]$ = maximum value considering items 1..i with capacity w. Cells shaded green contributed to the optimal solution.

Item \ w	0	1	2	3	4	5	6	7	8
0 (none)	0	0	0	0	0	0	0	0	0
1 (w=2,v=3)	0	0	3	3	3	3	3	3	3
2 (w=3,v=4)	0	0	3	4	4	7	7	7	7
3 (w=4,v=5)	0	0	3	4	5	7	8	9	9
4 (w=5,v=6)	0	0	3	4	5	7	8	9	10

4.3 Reading the Answer

$K[4][8] = 10 \rightarrow$ Maximum value achievable = **10**

4.4 Traceback — Which Items Were Selected?

Starting at $K[n][W]$, compare each row with the row above. A difference means the item was selected.

Item	Weight	Value	Selected?	Cum. Value
Item 1	2	3	No	—
Item 2	3	4	YES	4
Item 3	4	5	No	—
Item 4	5	6	YES	10
TOTAL	8	10		

Optimal selection: Item 2, Item 4 | Total weight: **8** / 8 | Total value: **10**